"Evaluación Integradora – Algoritmos y Estructura de Datos - Primera parte".

Estudiante: Gentile, Agustín Raúl

DNI: 32,7097,735

Fecha entrega: 06/10/2026

Legajo: 199433

Gentile, Agustín Raúl (DNI: 32709735) (Legajo: 199433)

Contexto:

Plataforma de análisis de incidentes y rutas

Una organización procesa eventos (incidentes) que contienen los siguientes campos: id, timestamp, categoría, prioridad, texto (descripción), origen y destino. Con esta información se busca desarrollar herramientas que permitan analizar los incidentes y apoyar la toma de decisiones.
En este marco, el sistema debe permitir:

- almacenar y consultar eventos de forma eficiente,
- buscar patrones en textos (alertas),
- priorizar atención (colas/heap),
- analizar red de rutas (grafos) y optimizar decisiones,
- estimar costos y justificar decisiones con complejidad y mediciones,
- y resguardar información sensible mediante encriptación (RSA a nivel demostrativo).

Objetivos:

La implementación debe realizarse en Python, utilizando las herramientas y enfoques trabajados durante el cursado, e incluir pruebas y mediciones reproducibles que permitan verificar el funcionamiento de las soluciones propuestas.

Formato de entrega

Requisito	**Detalle**
Informe académico	4 a 6 páginas
Entrega de código	Repositorio o carpeta comprimida con estructura modular
Integración de conceptos	Módulo 1: Python + POO + diseño modular + docstrings
Integración de conceptos	Módulo 2: Estructuras lineales + TDA + Big-O + time/timeit
Integración de conceptos	Módulo 3: Búsqueda, ordenamiento, hashing, BST

Entregables mínimos

1. **Diseño del sistema (POO + modularidad)**
   a. Definir una arquitectura mínima con clases y módulos: Event, EventStore, Index, Router, TextAnalyzer.
   b. Incluir docstrings y breve justificación de decisiones de diseño.

2. **Estructuras lineales (TDA + operaciones)**
   a. Implementar y usar al menos dos:
      i. Stack o Queue (con collections.deque), y
      ii. PriorityQueue (con heapq) para priorizar incidentes por severidad/tiempo.
   b. Explicar operaciones y complejidad.

3. **Búsqueda/ordenamiento y medición**

a. Implementar y comparar:
    i. búsqueda secuencial vs binaria (con bisect para consulta indexada),
    ii. dos ordenamientos entre: inserción / selección / burbuja / quicksort / mergesort,
    iii. comparación con sorted() o list.sort().
b. Medir con timeit sobre tamaños crecientes y analizar resultados (tiempo y, si aplica, memoria).

4. **Hashing e índices**
a. Diseñar un índice por clave (por ejemplo, id, categoría o origen) usando: diccionarios de Python y/o tabla de dispersión implementada (opcional avanzada).
b. Explicar colisiones (conceptualmente) y justificar elección.

5. **Síntesis integradora (1 página) Responder:**
a. ¿Qué decisiones de estructura/algoritmo mejoraron el rendimiento y por qué?
b. ¿Dónde aparece el trade-off "tiempo vs memoria" en el caso?
c. ¿Cómo se conectan POO + modularidad + estructuras + medición para construir software mantenible?

6. **ANEXO obligatorio:** Registro del uso de IAG según indicaciones generales (diálogos, fecha, herramienta y reflexión ética).

Gentile, Agustín Raúl (DNI: 32709735) (Legajo: 199433)

Introducción

El análisis y diseño del sistema se basa en los conceptos introducidos en los módulos 1,2 y 3. Se implementa una arquitectura modular basada en el paradigma de programación orientado al objeto, modularizando el sistema con clases con responsabilidad única.

La implementación incluye tanto estructuras de datos lineales: ***listas, pilas, cola, cola prioridad de atención,*** como estructuras jerárquicas: ***árbol y grafos, diccionarios (índice por hashing)***, mediciones de tiempo con timeit, que respalda las decisiones de diseño y selección de las estructuras implementadas.

La modularidad nos permite separar responsabilidades por archivo, lo que facilita las pruebas unitarias, la lectura y el versionado. Además, permite en un futuro cambiar la estructura de datos seleccionado sin afectar el resto del programa. Todos los módulos incluyen "docstrings" para la documentación de los módulos y métodos.

Se implementa la clase ***"Event"*** la cual representa la entidad principal del sistema (eventos), una clase ***"EventStore"*** destinada al almacenamiento tanto de eventos histórico, eventos atendidos y prioritarios. El manejo de índices se implementa en una clase ***"Index"***, nos facilita mantener múltiples índices, indexando según una clave(llave). Se implementa una clase para el análisis de texto, búsqueda de patrones, la misma se encuentra en el modulo ***"TextAnalizer"***. La clase ***"Router"*** modela la red de rutas entre incidentes entre los nodos.

Decisiones de diseño: Los eventos se cargan tanto a la cola de históricos como la cola de prioridad en el orden en el que llegan, además se va armando la ruta de incidentes. Al atender un evento, se deja en la cola de eventos históricos y el mismo va a parar a la pila de atendidos y se eliminan de la cola de prioridades y de la red de rutas, no se quita de ningún índice ni de la estructura destinada al análisis de texto. Con los históricos nos permite llamar a los métodos de búsqueda y ordenamiento; y permitir el análisis de tiempo y efectividad de los mismos.

Definición de la Clase y Responsabilidades:

IncidentService:

Orquestador del sistema, maneja y coordinado el flujo de operaciones del sistema invocando al resto de las clases.

Event:

Entidad del mundo real, modela cada evento mediante una clase; definiendo sus atributos y métodos para manipularlo.

Gentile, Agustín Raúl (DNI: 32709735) (Legajo: 199433)

EventStore:

Es el almacén de los eventos del sistema. Implementa una cola de llagada la misma almacena el histórico de los eventos, una pila de eventos atendidos, y una cola de prioridades destinada a atender los eventos prioritarios. Estructuras implementadas:

- ***Cola llegada:*** implementada con collection.deque. Guarda los eventos en el orden en que llegaron. Operaciones como append y popleft O(1)
- ***Cola Prioridades:*** Implementada con heap (max-heap); La prioridad de atención se da por la tupla (-prioridad, timestamp, evento), gracias a la incorporación de un max-heap se atiene el evento de más prioridad.
- ***Pila Atendidos:*** Implementada con collection.deque. Opera como LIFO, manejarla como una pila podemos deshacer las atenciones en el orden en el que fueron atendidas. Las operaciones de append y pop se realizan en O(1).

Index:

Maneja indices de sistema, nos permite consultar los eventos por determinada llave. Imprenta una tabla de hash basada en dict que mapea una clave (categoría, prioridad u origen).

- ***Llave:*** Nombre del indice o de la clave por el cual se indexa el diccionario.
- ***Tabla hash / Diccionario (Dict):*** Implementada en dict (hashing), Las operaciones de consulta y inserción se dan en promedio O(1).

Router:

Modela la red de rutas de incidentes gracias a la incorporación de un grafo no dirigido ponderado, lo cual nos permite establecer un vínculo entre cada nodo (ORIGEN/DESTINO) y la cantidad de incidentes que se dan entre ambos. Se implementa un método basado en BFS el cual nos otorga el camino con menos saltos, entendiendo menos salto como cada salto es atravesar un nodo, y no el número de incidente (ponderación).

Rsa:

Encriptación RSA solo a nivel demostrativo, encripta los textos de los eventos para mantener la información sensible protegida.

TextAnalizer: Analizador de texto. Permite responder consultas como: cantidad de frecuencias de una determinada palabra, o relaciones entre palabras; determinar en que eventos aparecen ciertas palabras.

- ***Diccionario de frecuencias:*** Implementa un diccionario con “dict” donde las llaves son las palabras, almacenan la cantidad de frecuencias de la palabra y un diccionario “dict” destinado a almacenar la cantidad de frecuencias dentro de cada evento.
- ***Grafo dirigido ponderado:*** Implementa un grafo dirigido ponderado. Establece la relación entre las palabras y la ponderación nos permite llevar el número de veces que aparece la relación en los textos de los eventos.

Gentile, Agustín Raúl (DNI: 32709735) (Legajo: 199433)

Búsqueda y ordenamiento:

Se implementa los métodos de ordenamiento: select_order, bubble_sort, sorted y .sort().; y los métodos de búsqueda: secuencial, binaria y binaria_bistect. Ademas se realizar mediciones de tiempo con timeit sobre una base de 50,, 100, 250, 500 y 1000 eventos.

Algoritmo / Método	Mejor Tiempo	Tiempo Promedio	Peor Tiempo
Busq. Secuencial	O(1)	O(n)	O(n)
Busq. Binaria	O(1)	O(n log n)	O(n log n)
Busq. Binaria Bisect	O(1)	O(n log n)	O(n log n)
Select (Selection Sort)	O(n^2)	O(n^2)	O(n^2)
Bubble (Bubble Sort)	O(n)	O(n^2)	O(n^2)
sorted() (Timsort)	O(n)	O(n log n)	O(n log n)
.sort() (Timsort)	O(n)	O(n log n)	O(n log n)

Medicoines:

N° de eventos	Búsqueda secuencial (s)	Búsqueda binaria (s)	Búsqueda binaria bisect (s)
50	0.00009706	0.00003741	0.00004441
100	0.00013169	0.00004612	0.00004226
250	0.00035182	0.00004426	0.00004406
500	0.00072854	0.00006272	0.00004896
1000	0.00143928	0.00007400	0.00005785

N° de eventos	sorted() (s)	list.sort() (s)	bubble_sort (s)	select_order (s)
50	0.00029784	0.00017648	0.00024265	0.00379343
100	0.00075214	0.00034792	0.00062236	0.01168201
250	0.00132164	0.00073047	0.00258623	0.06988263
500	0.00282847	0.00149047	0.00930494	0.35886413
1000	0.00610173	0.00316256	0.03752538	1.55817065

Síntesis Integradora:

- **¿Qué decisiones de estructura o algoritmo mejoraron el rendimiento?**

El heap binario para la cola de prioridad lleva la inserción y extracción de O(n) (lista ordenada) a O(log n).

Los índices por hashing llevan las consultas por categoría, prioridad u origen de O(n) en una lista a O(1) promedio.

Reemplazar list por deque para las colas evita el costo O(n) del pop(0) .

BFS sobre el grafo garantiza el camino mínimo en saltos en O(V + E).

El uso de implementar dict nativo, nos desentendemos de implementar una tabla de hashing y pensar en las colisiones, etc. La mima es una estructura super potente y nos ofrece el comportamiento esperado con rendimiento optimizado .

- **¿Dónde aparece el trade-off tiempo vs memoria?**

Cada índice consume memoria proporcional a N, pero reduce el costo de consulta de O(n) a O(1). Es una inversión de memoria para ganar tiempo.

El TextAnalyzer mantiene simultáneamente un diccionario de frecuencias y un grafo de concurrencias: duplica el costo en memoria pero cada tipo de consulta se responde en su estructura óptima.

- **¿Cómo se conectan POO, modularidad, estructuras y medición?**

La POO permitió modelar el sistema en módulo (clases) los cuales tienen una responsabilidad única. La modularidad permite y facilita leer, testear y modificar cada pieza de forma aislada. La elección de estructuras no se hizo por intuición, sino con base en análisis de complejidad y confirmada por mediciones empíricas con timeit. El resultado es software altamente mantenible y escalable, además nos permite modificar cualquier estructura sin afectar el resto del programa.

Diálogos con IA (claude): Los diálogos relevantes para el diseño y desarrollo se encuentran en un archivo “dialogosIA.txt”.